# コンピュータビジョン特論A/B
# 現代脳計算論

# Computer Vision A/B
# Modern Neural Computation

Keisuke Yoneda, Kanazawa University
Email: k.yoneda@staff.kanazawa-u.ac.jp

# Image Stitching

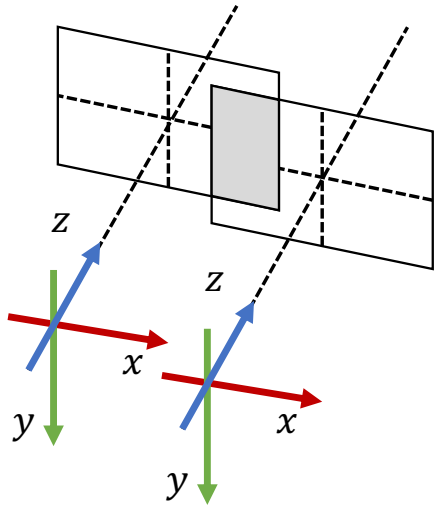


- Feature matching
  - Find Corresponding pixels
  - Compute Homography Matrix
    - $\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = H \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{12} & h_{21} & h_{22} \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$

# Camera Model based Image Stitching



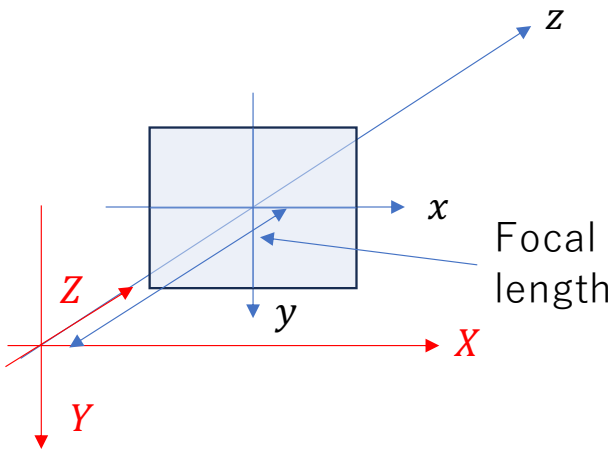
- Image stitching for multiple cameras based on camera parameters
  - If the camera positions are fixed, …
    - Corresponding pixels can be computed based on intrinsic parameters and extrinsic parameters
    - Pixel tables can be computed in advance to reduce computational time



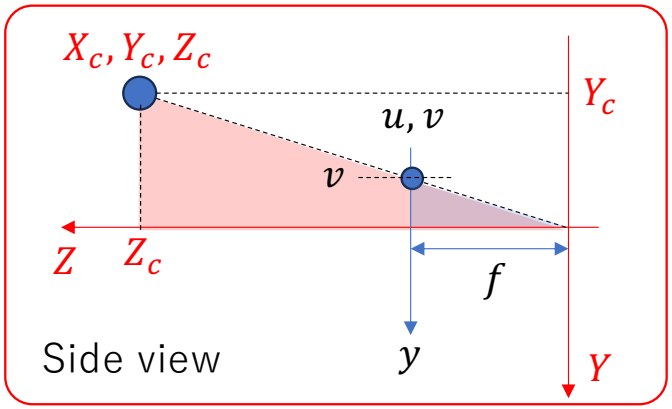
# Pin-hole Camera Model (1/2)



- Pinhole camera model
  - Compute 3D point to 2D image plane




$x - y - z$: Image Coordinate
$X - Y - Z$: Camera Coordinate




$$u' = f\frac{X_c}{Z_c}, v' = f\frac{Y_c}{Z_c}$$

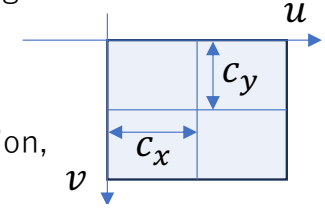
When the origin of the image coordinate is the center of the image

$$u = f_x\frac{X_c}{Z_c} + c_x, v = f_y\frac{Y_c}{Z_c} + c_y$$

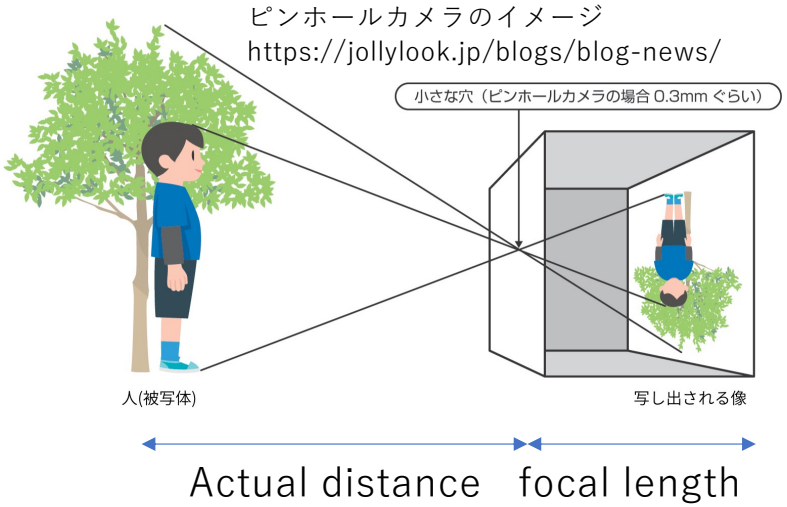
Without considering camera lens distortion

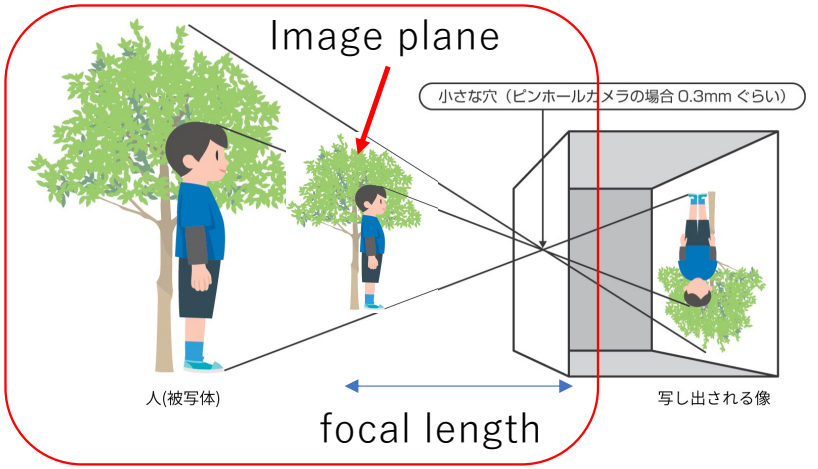
$$c_x = \frac{w}{2}, c_y = \frac{h}{2}$$

$w$: horizontal resolution,
$h$: vertical resolution




ピンホールカメラのイメージ
https://jollylook.jp/blogs/blog-news/

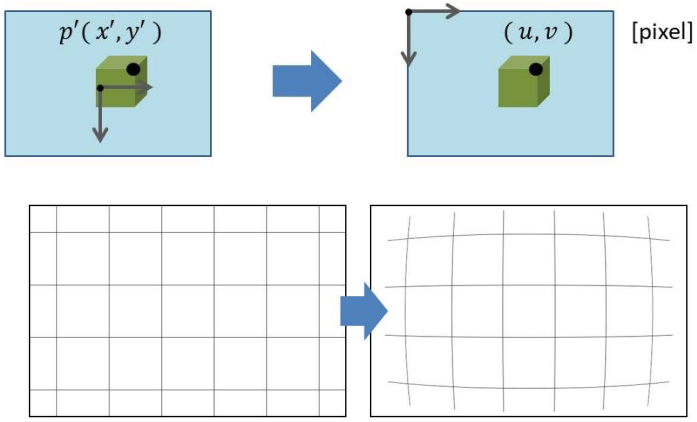


# Pin-hole Camera Model (2/2)



- Compute 3D point $P = [X, Y, Z]^T$ to image pixel $[u, v]^T$
  - $$s\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_2 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \underbrace{[\boldsymbol{R} \quad \boldsymbol{t}] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}}_{x, y, z}$$
    - $f_*$: focal length, $c_*$: principal point (intrinsic parameters)
    - $\boldsymbol{R}, \boldsymbol{t}$: rotation matrix and translation vector (extrinsic parameters)
- Considering lens distortion
  - $x' = x/z, y' = y/z$
  - $$x'' = \frac{x'\left(1+k_1r^2+k_2r^4+k_3r^6\right)}{1+k_4r^2+k_5r^4+k_6r^6} + \left(2p_1x'y' + p_2\left(2x'^2 + r^2\right)\right)$$
  - $$y'' = \frac{y'\left(1+k_1r^2+k_2r^4+k_3r^6\right)}{1+k_4r^2+k_5r^4+k_6r^6} + \left(p_1\left(2y'^2 + r^2\right) + 2p_2x'y'\right)$$
    - $r^2 = x'^2 + y'^2$
    - $k_*, p_*$: camera distortion parameters (intrinsic parameters)
  - $u = f_x x'' + c_x, v = f_y y'' + c_y$



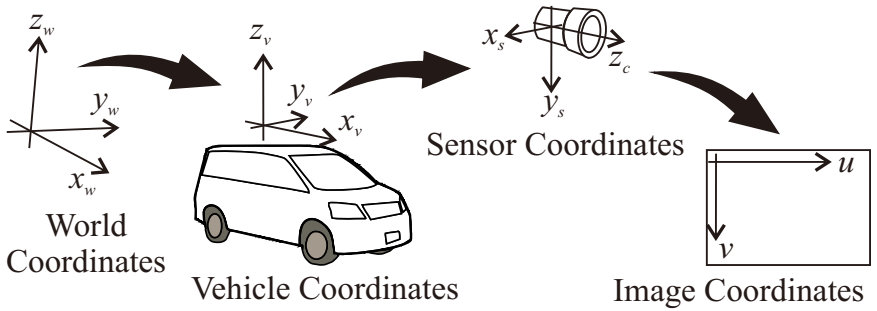

http://www.sanko-shoko.net/note.php?id=g3ft

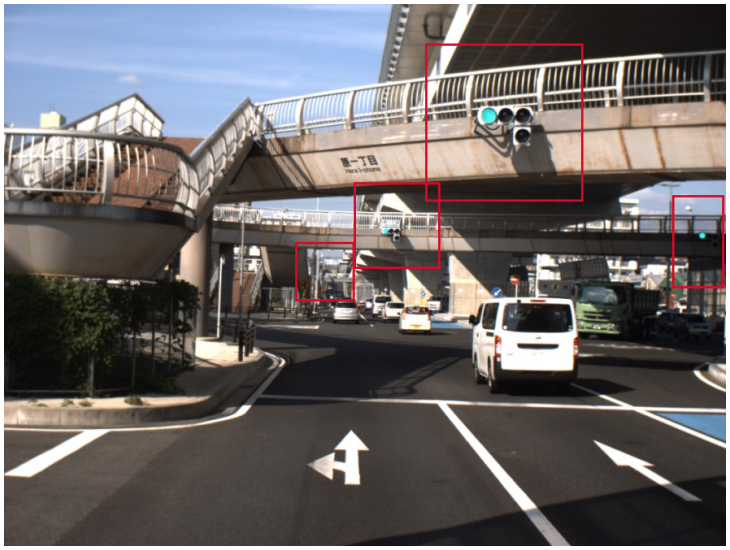
http://opencv.jp/opencv-2.2/cpp/calib3d_camera_calibration_and_3d_reconstruction.html

# Application of Camera Model (1/2)



- Traffic Light Recognition based on Digital Map
  - In automated driving, traffic lights should be recognized using camera image
  - Digital Map contains traffic light positions. It can be used to reduce false recognition and computational time
- Compute Global Position to Image Coordinate
  1. Global Coordinates -> Vehicle Coordinates
  2. Vehicle Coordinates -> Sensor Coordinates
  3. Sensor Coordinates -> Image Coordinates

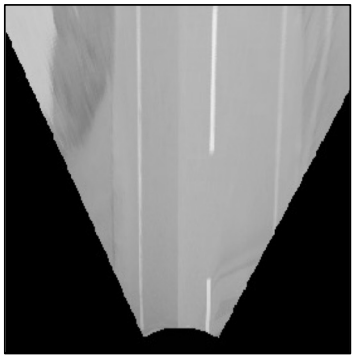


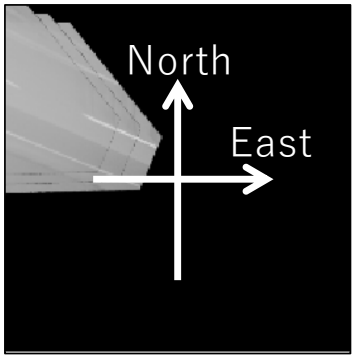



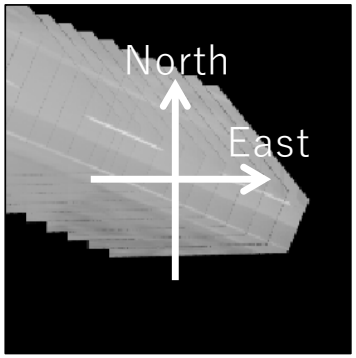
# Application of Camera Model (2/3)



- Based on camera parameters and X, Y, Z positions, the captured image can be converted to the image with arbitrary viewpoint.





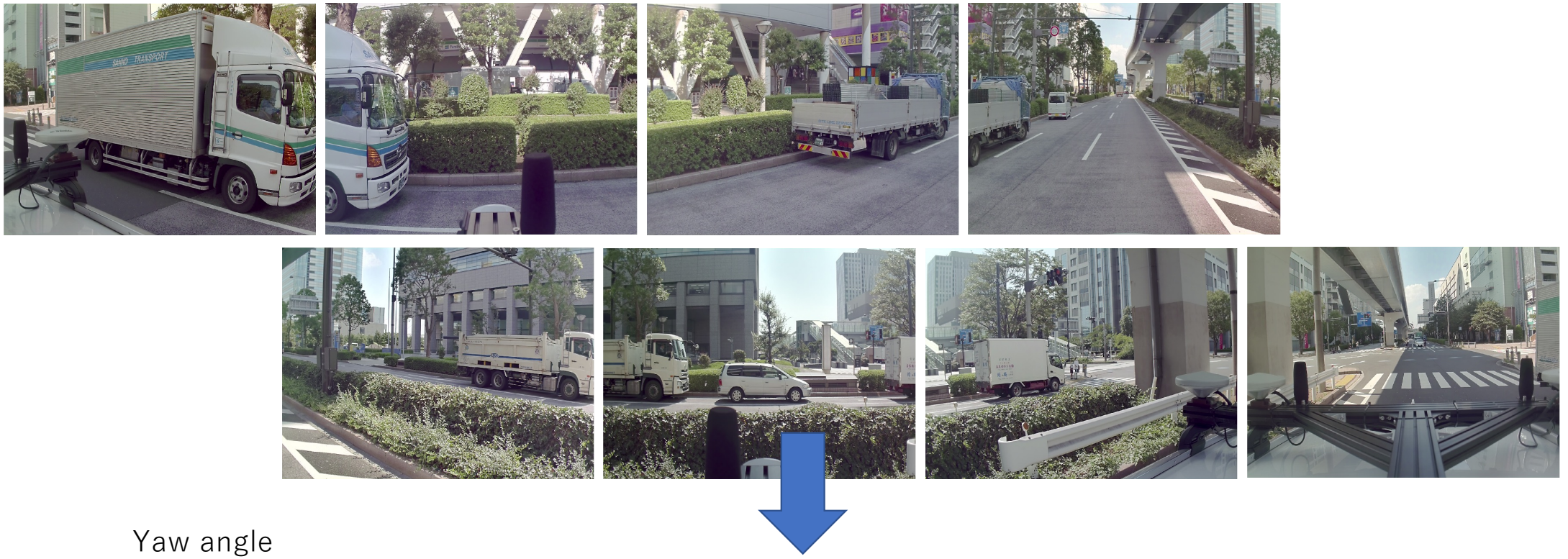




Generate Surrounding Image like Satellite image

Captured Image → Bird View Image → Convert & Accumulate Images using vehicle angle & position

# Application of Camera Model (3/3)

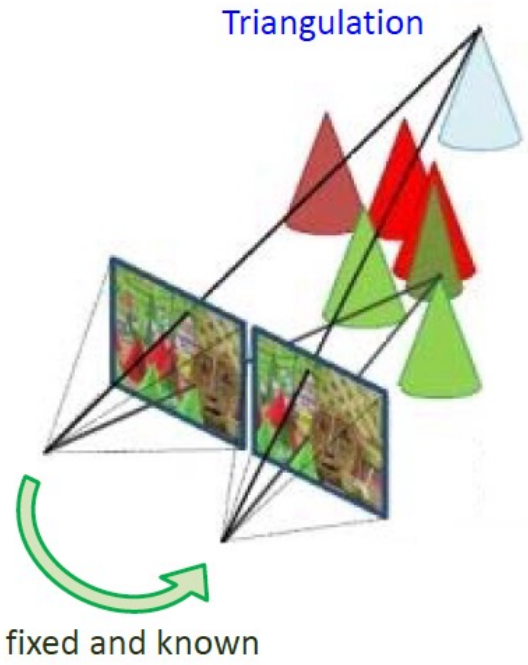


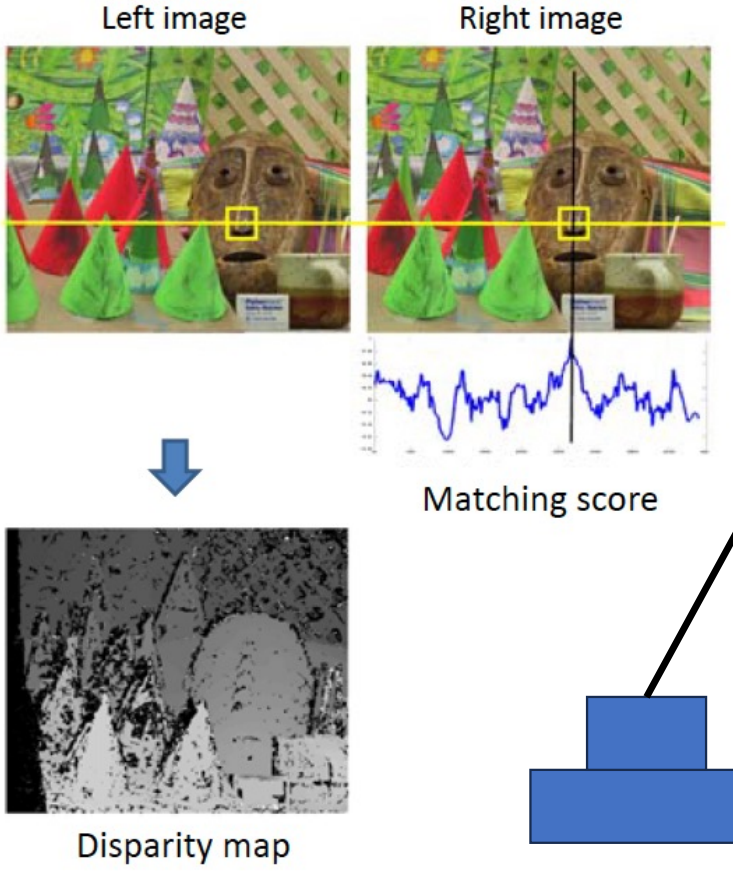


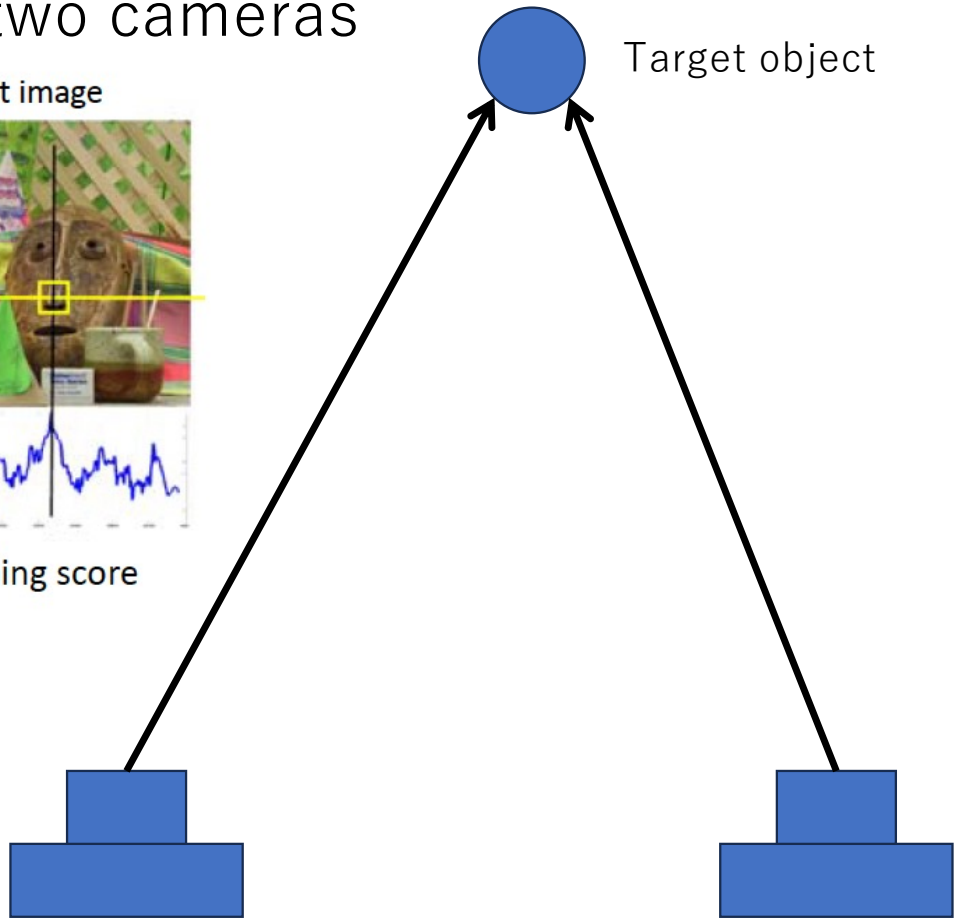
# Stereo Vision



- Compute Depth information using two cameras



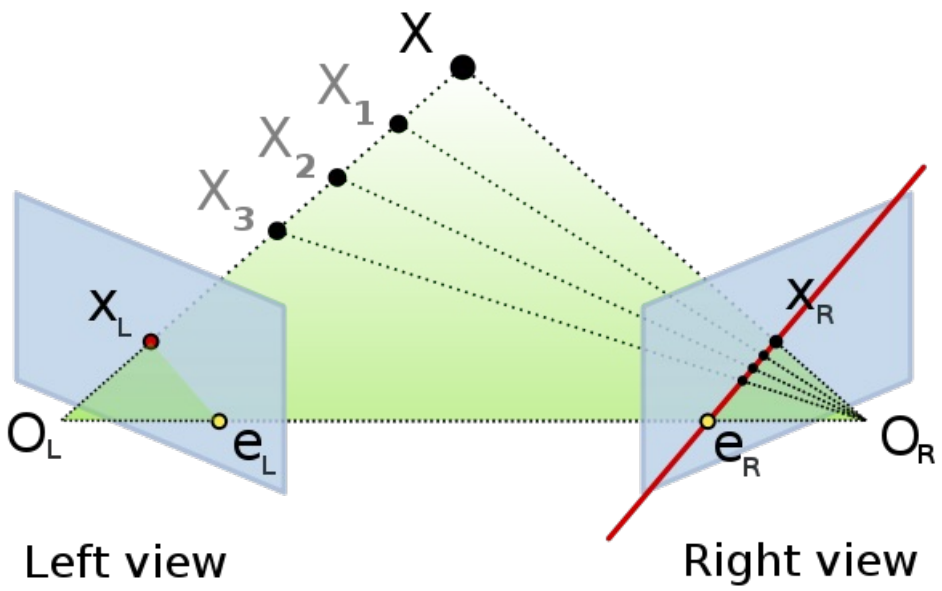



# Epipolar Geometry



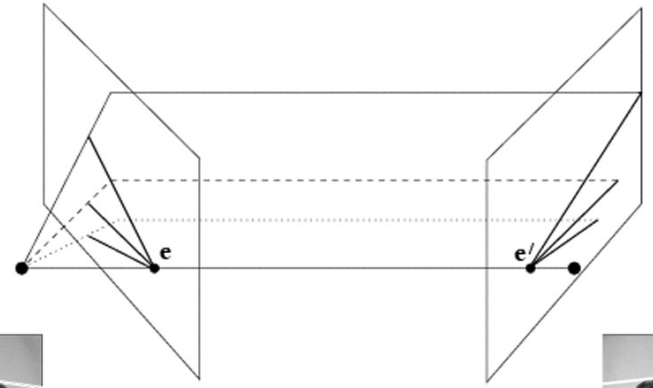
- Focus on object $X$ in left image,
  - Object $X$ can be seen at pixel $X_L$
    - Cannot obtain distance information using single image
    - But relative object direction $O_L - X$ can be obtained
- Epipolar Plane
  - Green Plane: $O_l X O_R$
- Epipolar Line in right image
  - Intersect line for Epipolar plane & Image plane for right camera
  - Red Line : $e_R X_R$
  - The object $X$ should be seen on the epipolar line




https://ja.wikipedia.org/wiki/%E3%82%A8%E3%83%94%E3%83%9D%E3%83%BC%E3%83%A9%E5%B9%BE%E4%BD%95

# Epipolar Geometry, Rectification





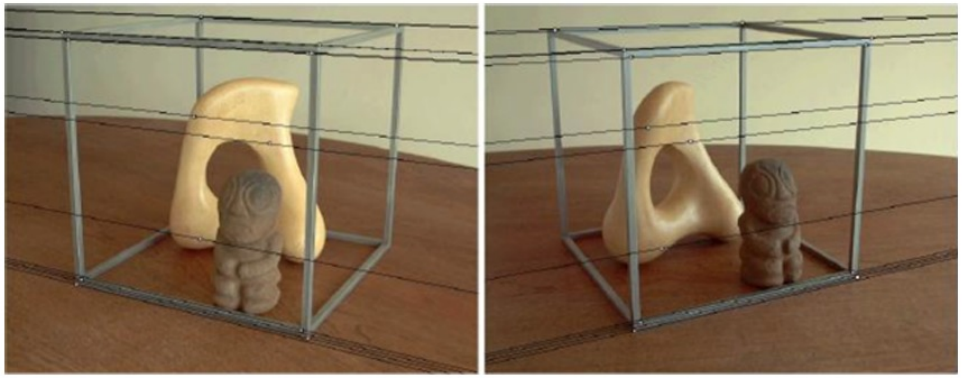
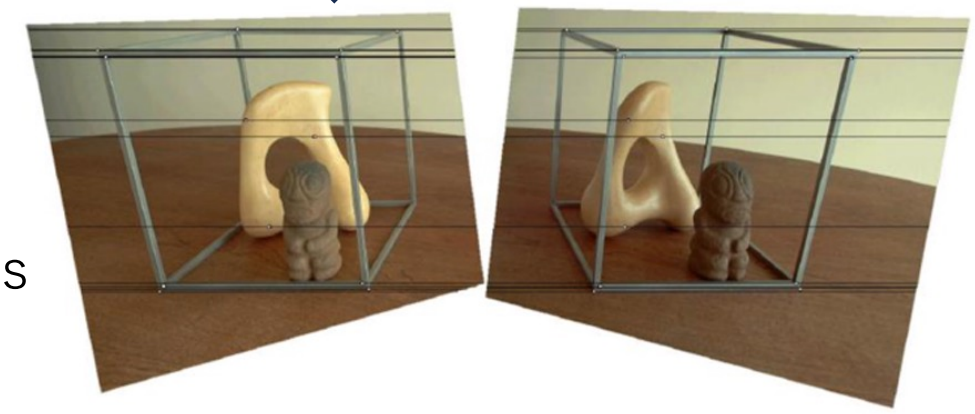



Image Rectification

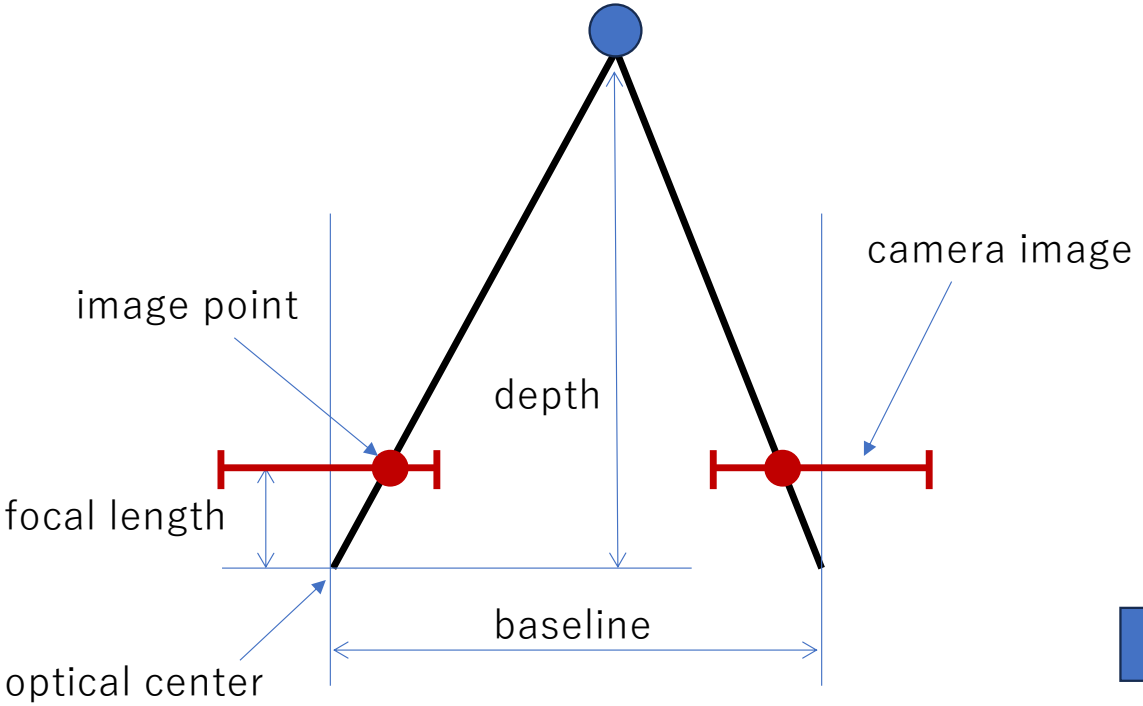


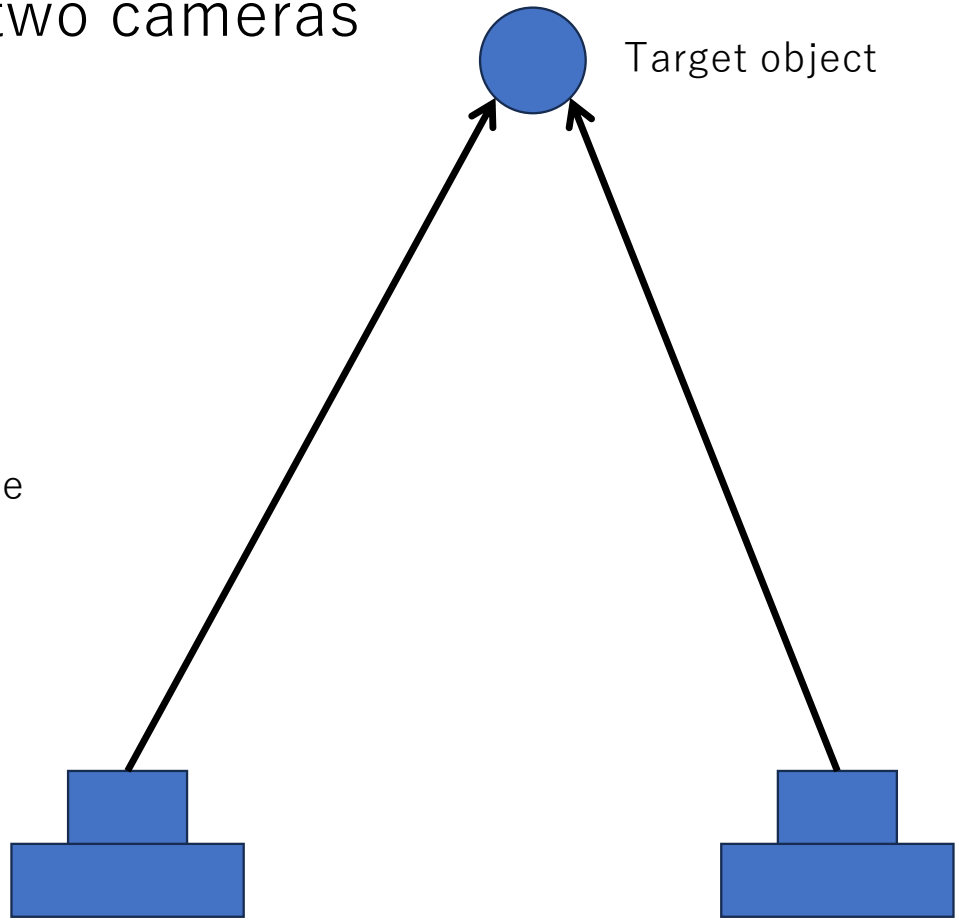
For stereo matching, horizontal epipolar lines are easier to find the corresponded pixel

# Stereo Vision



- Compute Depth information using two cameras

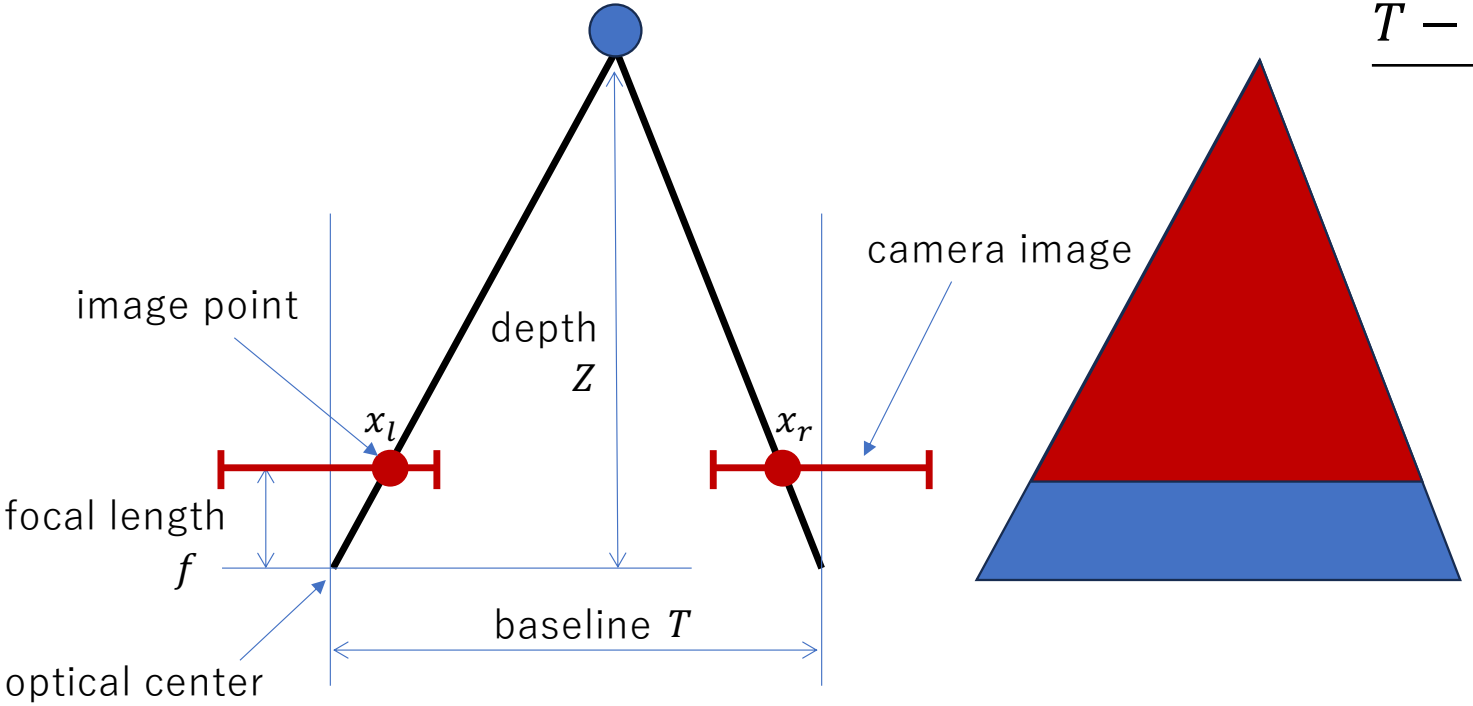


# Stereo Vision



- Compute depth using disparity value
  - Similar Triangles（三角形の相似）






$$T - |x_r - x_l| : T = Z - f : Z$$

$$Z(T - |x_r - x_l|) = T(Z - f)$$

$$\frac{T - |x_r - x_l|}{T} = \frac{Z - f}{Z} = 1 - \frac{f}{Z}$$

$$\frac{f}{Z} = 1 - \frac{T - |x_r - x_l|}{T}$$

$$= \frac{|x_r - x_l|}{T}$$

$$Z = f\frac{T}{|x_r - x_l|}$$

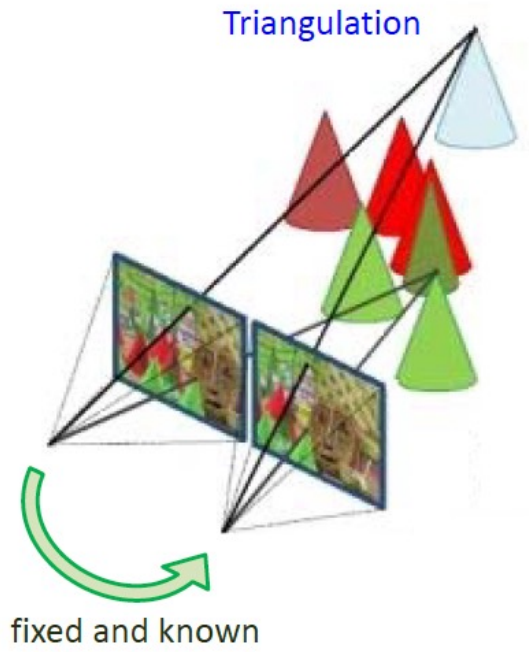
$Z$ : depth

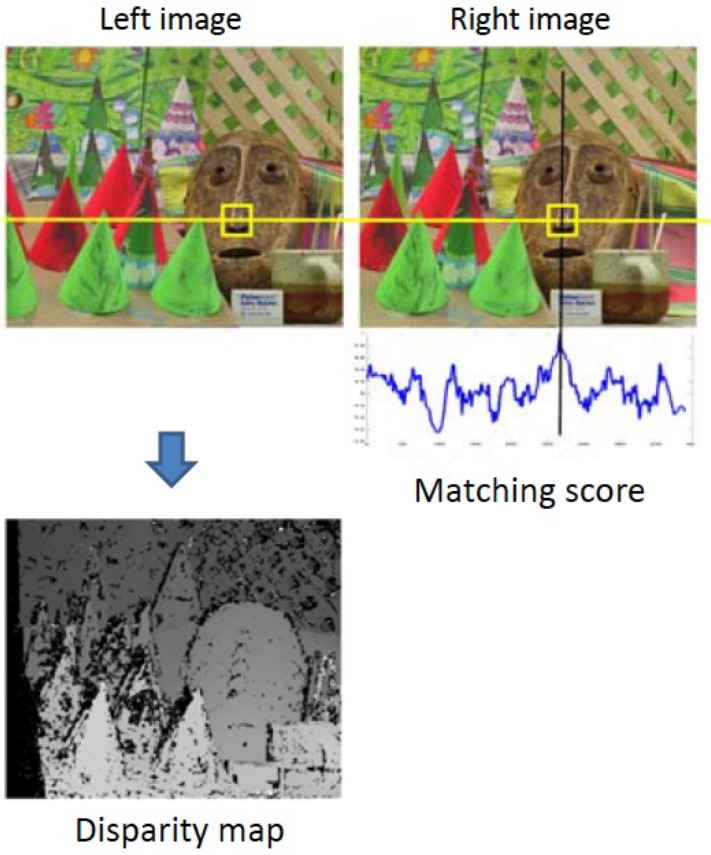
$x_r - x_l$: disparity（視差）

# Stereo Matching



- Compute depth value for each pixel
  1. Rectify Images
  2. For each pixel
     1. Find epipolar line
     2. Find corresponded pixel
     3. Compute depth value
        - $Z = f\frac{T}{d}$

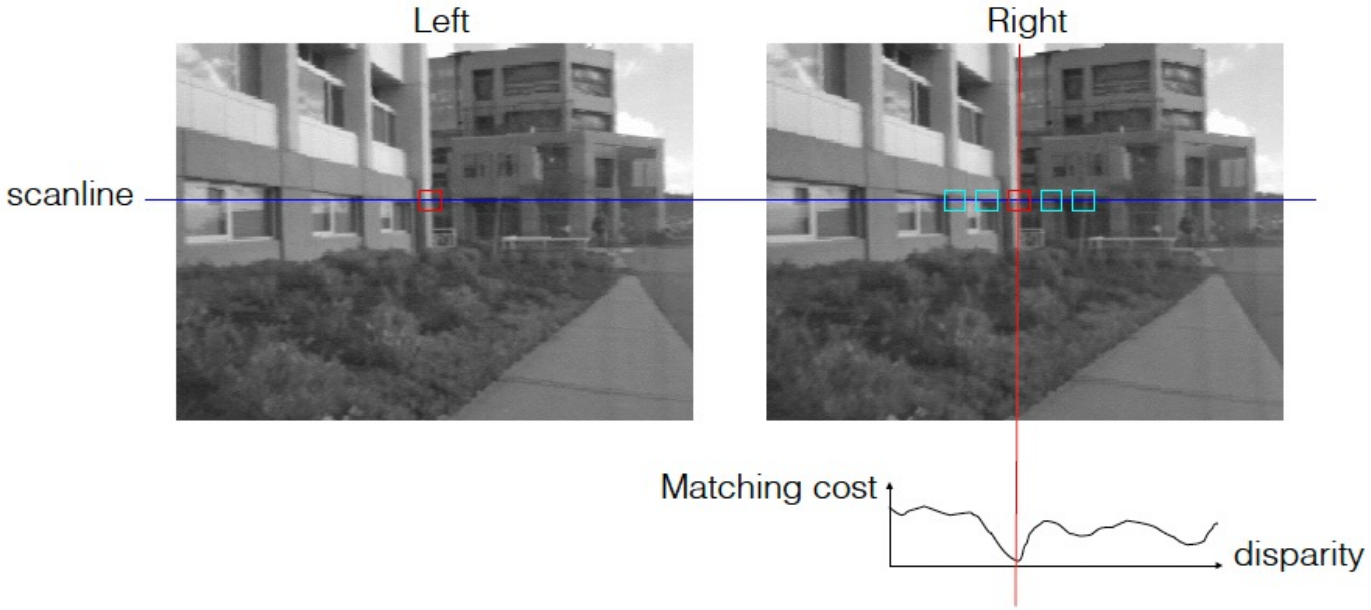


# Stereo Matching: Block Matching (BM)



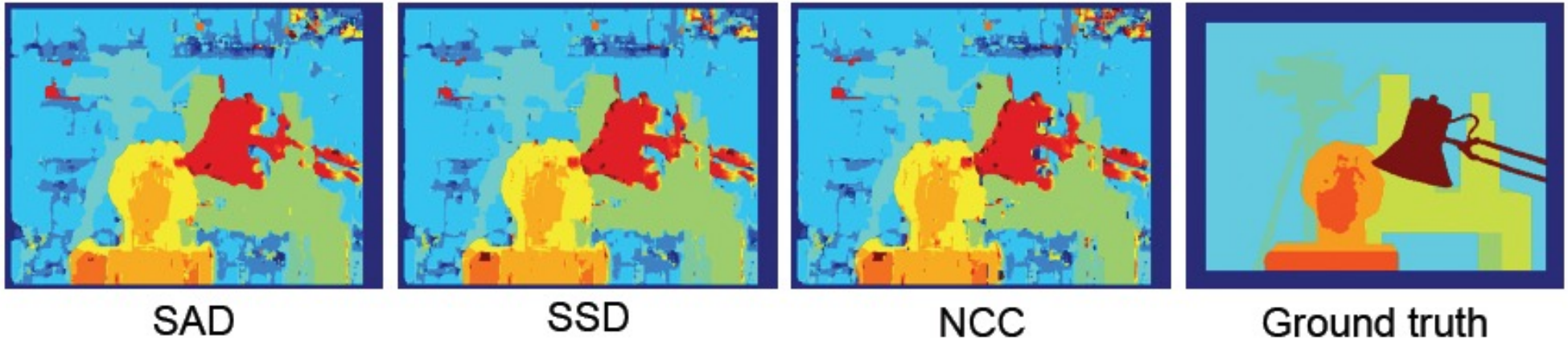
- Slide a window along the epipolar line
  - Compare contents of that window with the reference window in the left image
  - Find the best matched position
  - Matching cost: SSD or Normalized correlation (Like Template Matching)



# Stereo Matching: Block Matching (BM)



- Slide a window along the epipolar line
  - Compare contents of that window with the reference window in the left image
  - Find the best matched position
  - Matching cost: SSD or Normalized correlation (Like Template Matching)



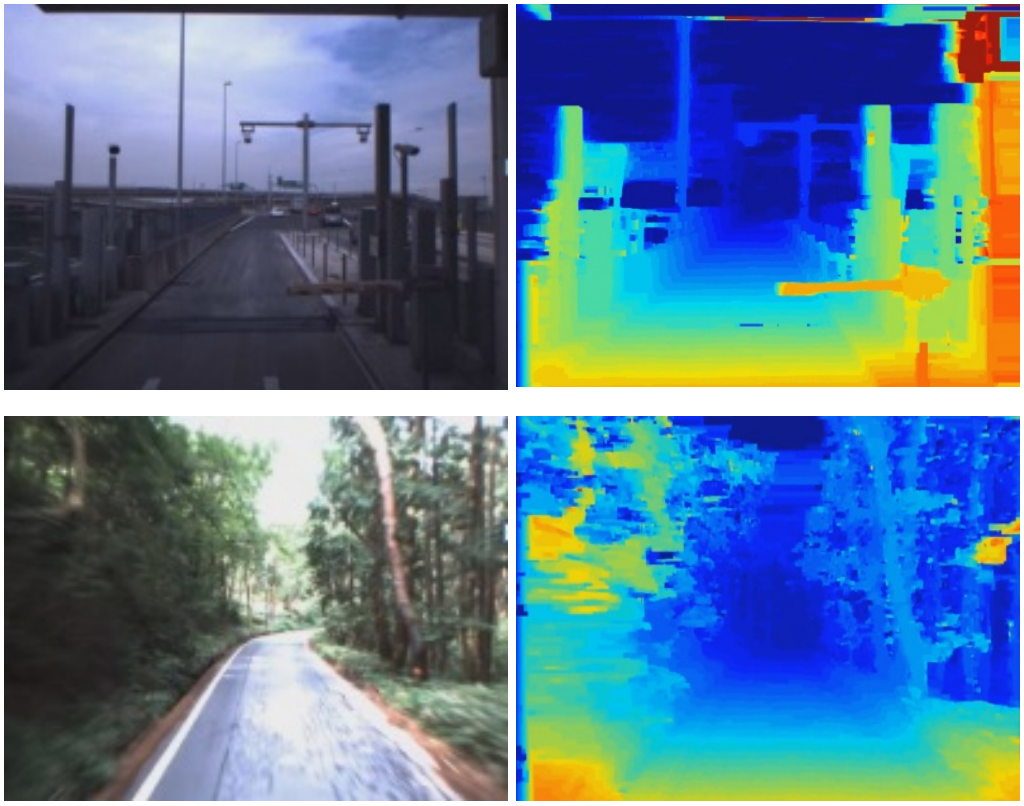

- Semi-Global Block Matching (SGBM)
  - Block Matching (Matching quality) + Smoothness cost

# Stereo Vision for Autonomous Driving



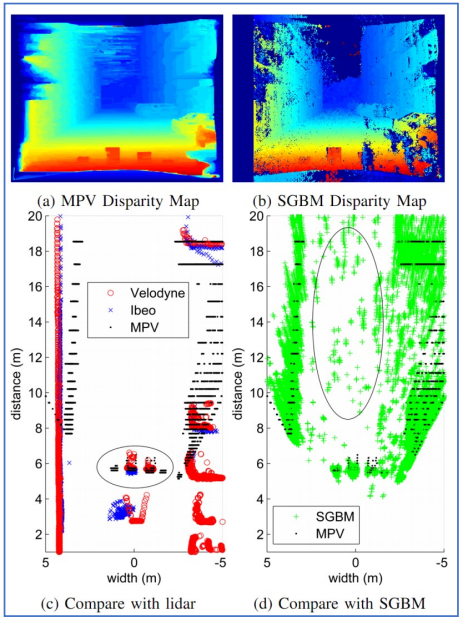
# Stereo Vision for Autonomous Driving
# Road Surface Recognition

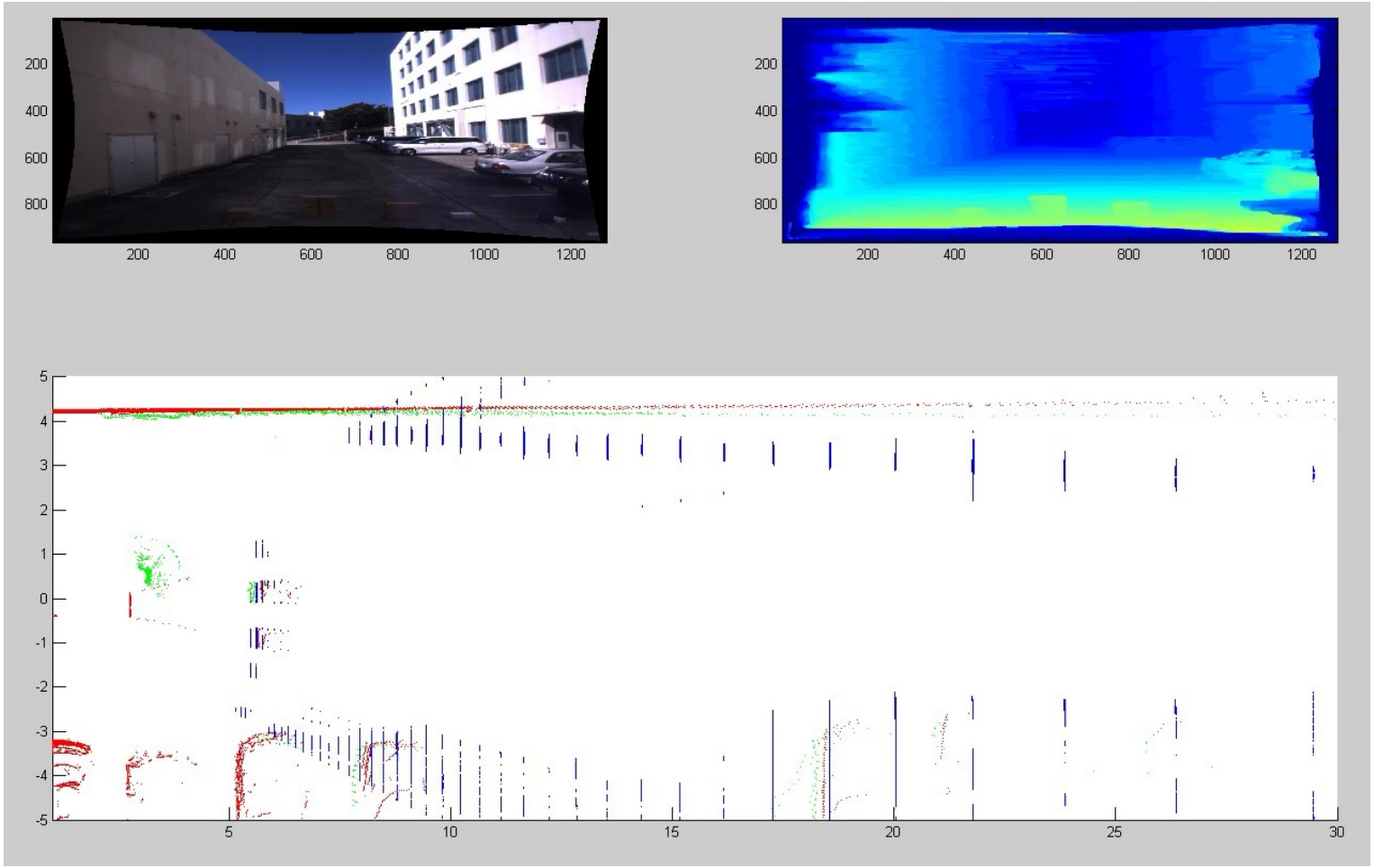




Road Surface Recognition
[Q. long, Q. Xie, S. Mita, K. Ishimaru, N. Shirai, IEEE ITSC(2014)]

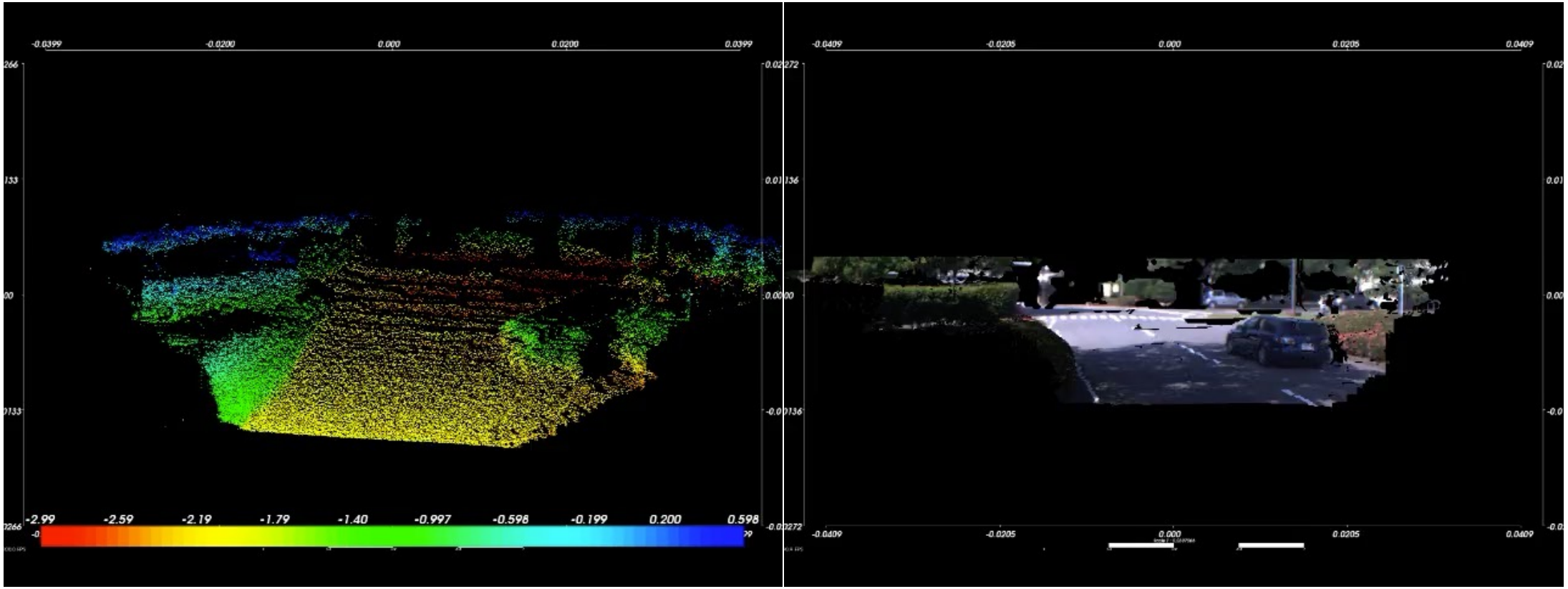
# Stereo Vision for Autonomous Driving
# Small Object Detection

# Stereo Vision for Autonomous Driving
# 3D Reconstruction





# Supervised Learning



- Machine Learning
  - Supervised Learning
- Perceptron
  - Multilayer Perceptron
  - Backpropagation
- Supervised Learning
  - Support Vector Machine
  - AdaBoost
  - Random Forest

# Machine Learning?

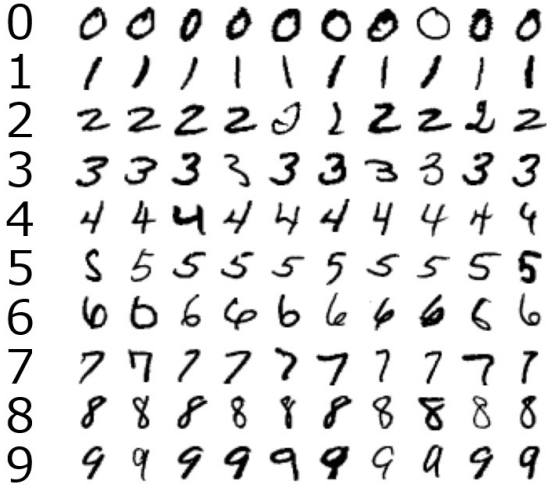


- Training a Task
  - E.g. Classification Task: Distinguish the category of input data
- Supervised Learning
  - Given: Set of input (x) and output (y)
  - Train function f (y = f(x))
- Unsupervised Learning
  - Given: input (x)
  - Learn relationship between input data
- Reinforcement Learning
  - Given: reward
  - Train policy function f (y=f(x)) maximizing reward

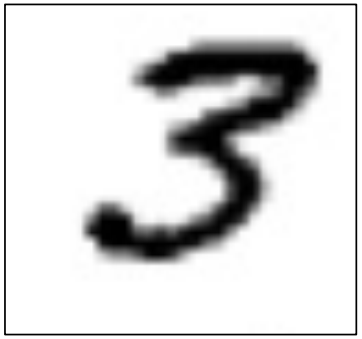
# Supervised Learning



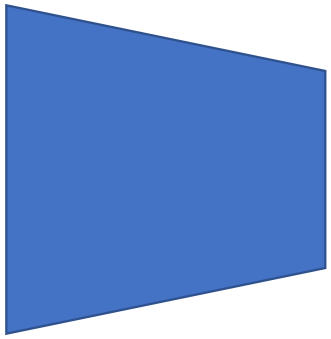
- Train mapping function for provided dataset
  - $y = f(x)$
    - $x$: input vector (n-dim)
    - $y$: output vector (m-dim)
    - $f(\cdot)$: mapping function




MNIST Dataset



input



mapping function
(algorithm)

task 0?, 1?, 2?, 3?, …

score [0.0, 0.05, 0.2, 0.7, …]

task even?, odd?

score [0.3, 0.7]

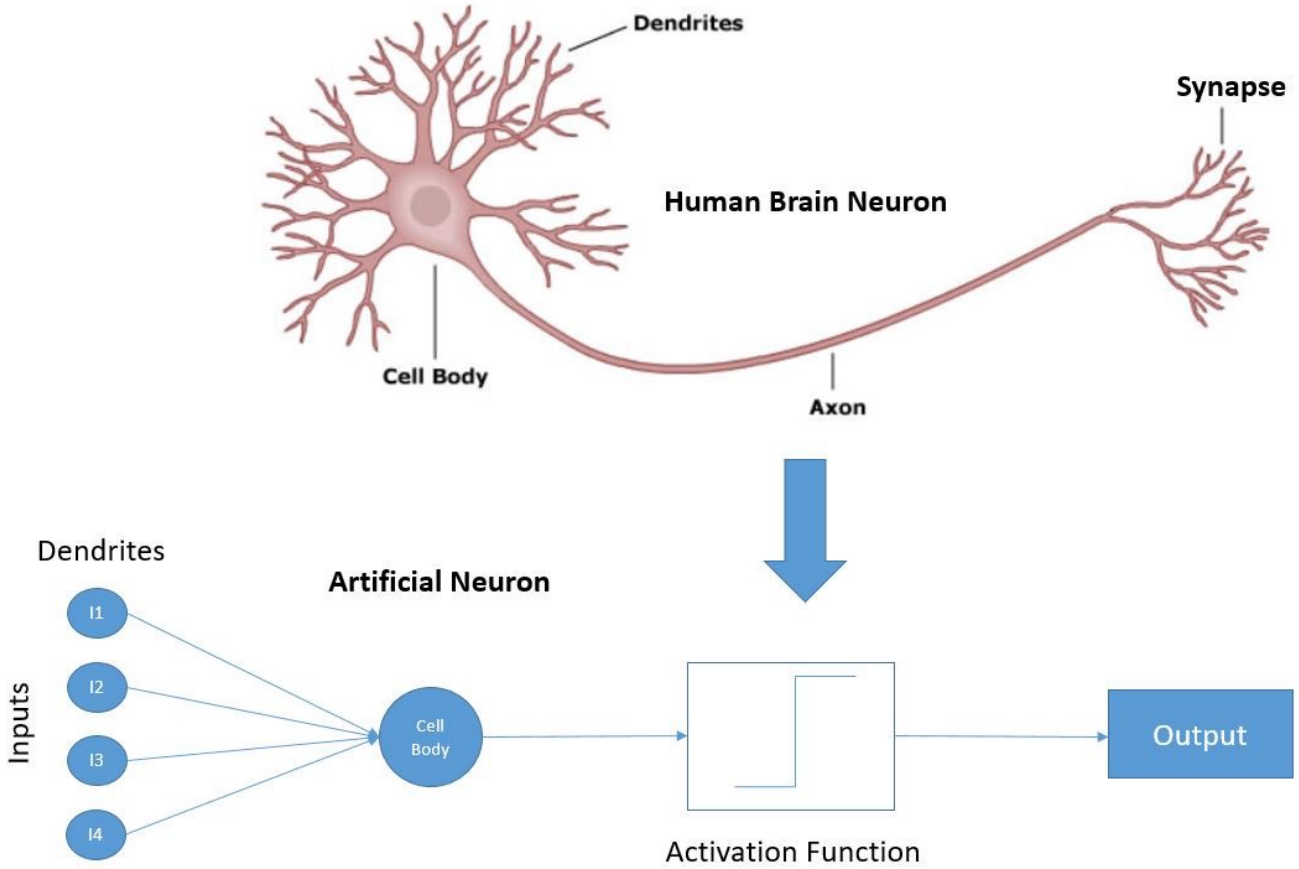
task 3?, or not?

score [0.9, 0.1]

output

# Perceptron






https://images.deepai.org/glossary-terms/cf6448554ec74f7fb58cf83192b6e904/synapse.jpeg

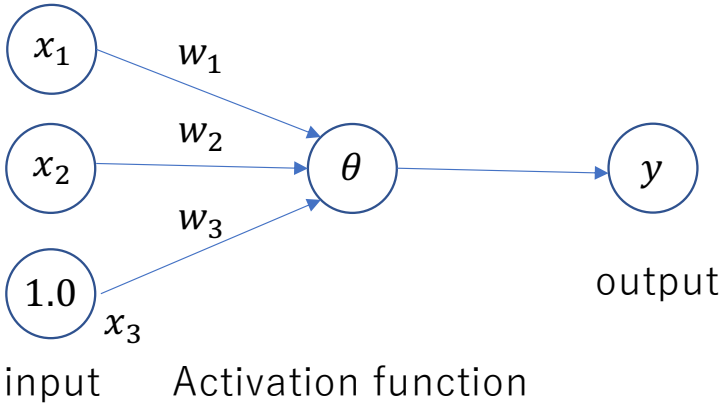
# Simple Perceptron



- Example: logical operation

| $x_1$ | $x_2$ | $y(AND)$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

| $x_1$ | $x_2$ | $y(OR)$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |




$$\theta(u) = \begin{cases} 1 & (u \geq 0) \\ 0 & (otherwise) \end{cases}$$

$$u = \sum x_i w_i$$

$$y = \theta(u)$$

How to determine $w_i$?

# Simple Perceptron



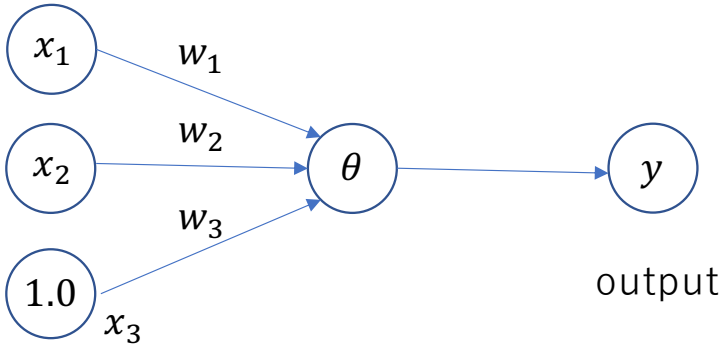
- Example: logical operation

| $x_1$ | $x_2$ | $y(AND)$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

| $w_1$ | $w_2$ | $w_3$ |
|---|---|---|
| 1.0 | 1.0 | -1.5 |

| $x_1$ | $x_2$ | $y(OR)$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

| $w_1$ | $w_2$ | $w_3$ |
|---|---|---|
| 1.0 | 1.0 | -0.5 |




$$\theta(u) = \begin{cases} 1 & (u \geq 0) \\ 0 & (otherwise) \end{cases}$$

$$u = \sum x_i w_i$$

$$y = \theta(u)$$

$(AND)$: $u = x_1 + x_2 - 1.5 = 0$

$\therefore x_2 = -x_1 + 1.5$

$(OR)$: $u = x_1 + x_2 - 0.5 = 0$

$\therefore x_2 = -x_1 + 0.5$

# Simple Perceptron



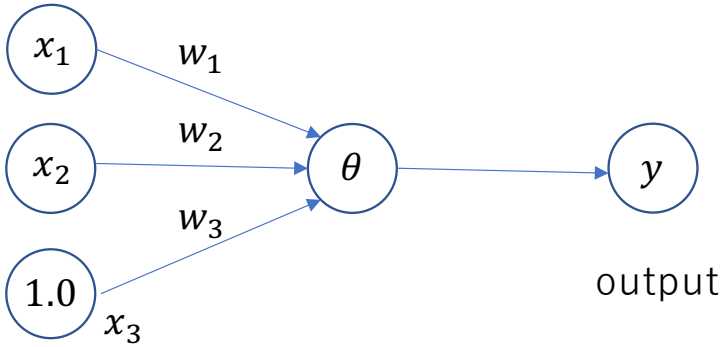
- Example: logical operation

| $x_1$ | $x_2$ | $y(AND)$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

| $w_1$ | $w_2$ | $w_3$ |
|---|---|---|
| 1.0 | 1.0 | -1.5 |

| $x_1$ | $x_2$ | $y(OR)$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

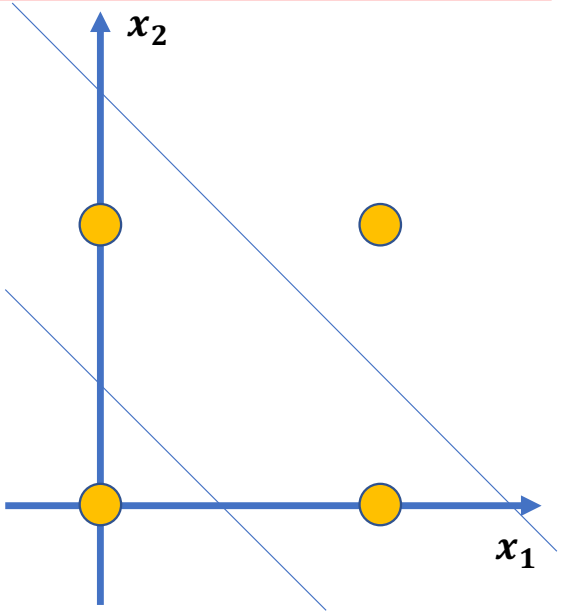
| $w_1$ | $w_2$ | $w_3$ |
|---|---|---|
| 1.0 | 1.0 | -0.5 |




$$\theta(u) = \begin{cases} 1 & (u \geq 0) \\ 0 & (otherwise) \end{cases}$$

$$u = \sum x_i w_i$$

$$y = \theta(u)$$




$(AND): u = x_1 + x_2 - 1.5 = 0$

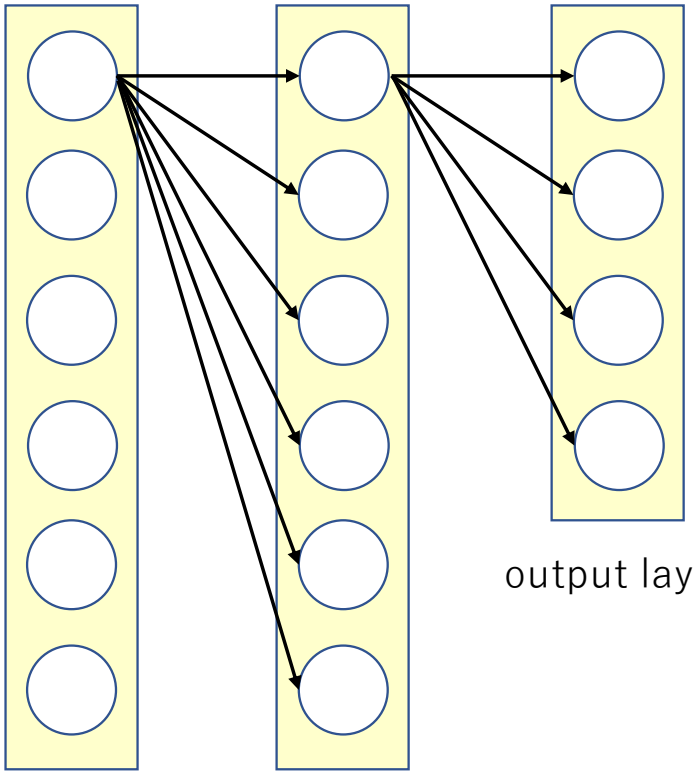
$\therefore x_2 = -x_1 + 1.5$

$(OR): u = x_1 + x_2 - 0.5 = 0$

$\therefore x_2 = -x_1 + 0.5$

# Multilayer Perceptron (1/2)

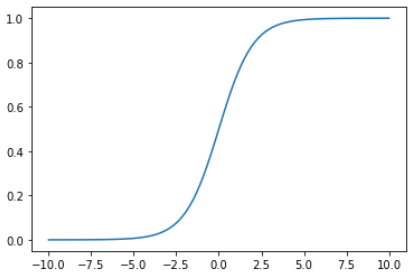




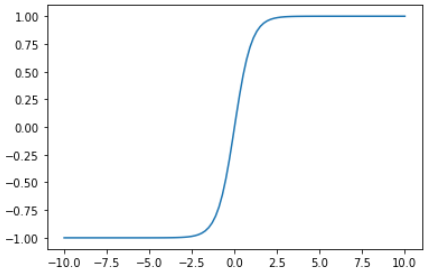
input layer hidden layer output layer

Activation function

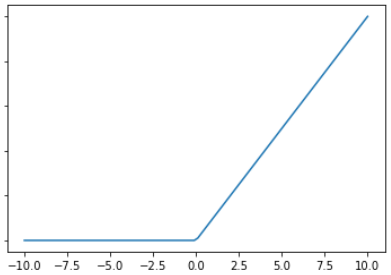
- Sigmoid
  - $\theta(u) = \frac{1}{1+e^{-u}} = \frac{\tanh\left(\frac{u}{2}\right)+1}{2} or \frac{1}{1+e^{-a(u+b)}}$
- Tanh
  - $\theta(u) = \tanh(u)$
- Relu
  - $\theta(u) = \max(0, u)$



sigmoid



tanh

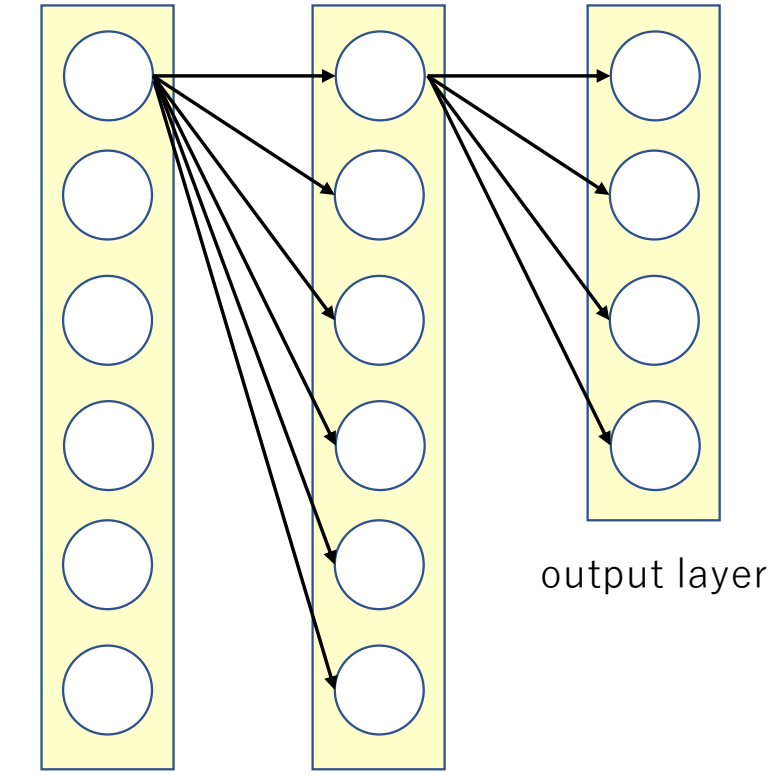


relu

How to optimize network parameters?

# Multilayer Perceptron (2/2)





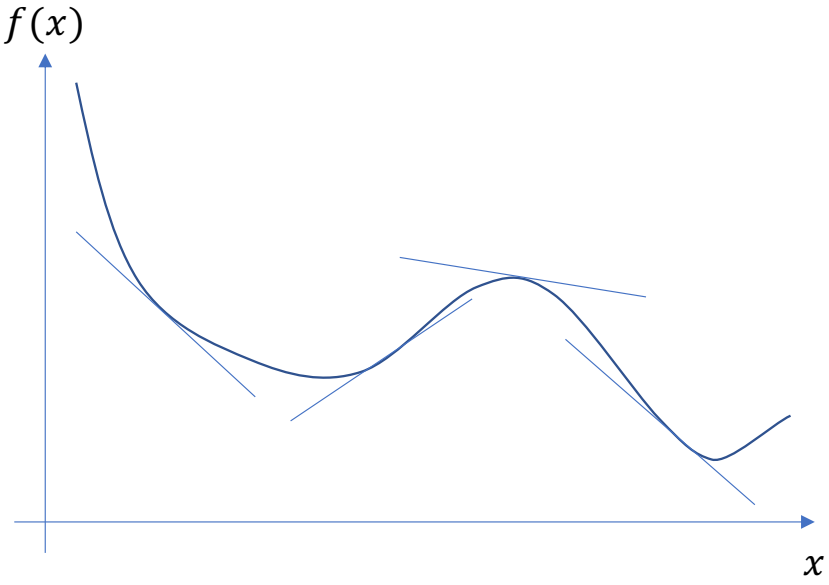

- Network Parameter
  - Weight
    - Input-Hidden: $N_i N_h$
    - Hidden-Output: $N_h N_o$
  - Bias value (optional)
    - Number of neurons: $N_i + N_h + N_o$
  - Activation function (optional)
    - Number of neurons: $(N_i + N_h + N_o) * N_a$

- Optimization method
  - Gradient descent (最急降下法)
    - Back-Propagation
  - Evolutionary computation (進化計算)
    - Genetic Algorithm
    - Particle Swarm Optimization
    - CMA-ES, …

# Gradient Descent (1/3)



- Find best $x$ with minimum cost
  - $x^{(k+1)} = x^{(k)} - \alpha f'(x^{(k)})$
    - $\alpha$: learning rate
    - $k$: iteration number
    - $f(x)$: cost function
  - Termination condition
    - $k > k_{max}$
    - $|f'(x^{(k)})| < \varepsilon$ (small value)
  - Need to obtain gradient value around $x$

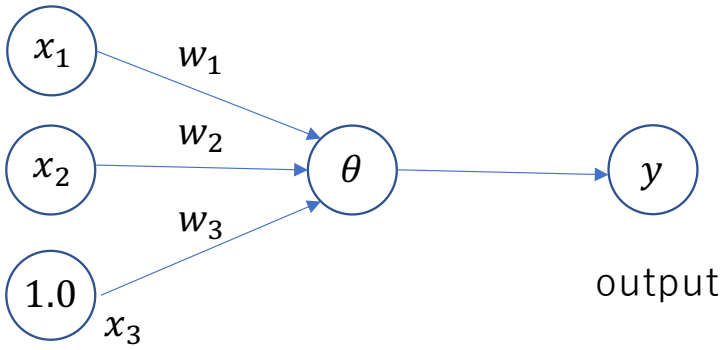



- Supervised learning
  - Train mapping function for provided dataset
  - $y = MLP(x)$
    - $x$: input vector (n-dim)
    - $y$: output vector (m-dim)
    - $\hat{y}$: label vector (m-dim)
  - How to comupte gradient value??

# Gradient Descent (2/3)



- Cost function for MLP (loss function)
  - Minimize following loss function
    - Mean Squared Error: $MSE = \frac{1}{n}\sum_i^n (\hat{y}_i - y_i)^2$
    - Mean Absolute Error: $MAE = \frac{1}{n}\sum_i^n |\hat{y}_i - y_i|$
    - Categorical Cross-Entropy, etc..
- How to compute the gradient?




$$\theta(u) = u$$

$$u = \sum x_i w_i$$

$$y = \theta(u) = x_1 w_1 + x_2 w_2 + x_3 w_3$$

$$E = (\hat{y} - y)^2 = (\hat{y} - x_1 w_1 - x_2 w_2 - x_3 w_3)^2$$

$$\frac{\partial E}{\partial w_i}?$$

# Gradient Descent (3/3)



- How to compute the gradient?

$$E = (\hat{y} - y)^2 = (\hat{y} - x_1 w_1 - x_2 w_2 - x_3 w_3)^2$$

$$\frac{\partial E}{\partial w_1} = 2(\hat{y} - x_1 w_1 - x_2 w_2 - x_3 w_3)(-x_1) = -2x_1(\hat{y} - x_1 w_1 - x_2 w_2 - x_3 w_3)$$

In case of $x_1 = 1.0, x_2 = 1.0, x_3 = 1.0, \hat{y} = 1.0, w_1 = 1.0, w_2 = 1.0, w_3 = 1.0$

$$\frac{\partial E}{\partial w_1} = -2 * 1.0 * (1.0 - 1.0 * 1.0 - 1.0 * 1.0 - 1.0 * 1.0) = (-2.0) * (-2.0) = 4.0$$

- Back Propagation: compute gradient using computational graph

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial a}\frac{\partial a}{\partial b}\frac{\partial b}{\partial c}\frac{\partial c}{\partial d}\frac{\partial d}{\partial e}\cdots\frac{\partial X}{\partial w_1}$$

e.g. $y = (ax + b)^2$

$$t = ax + b, y = t^2$$

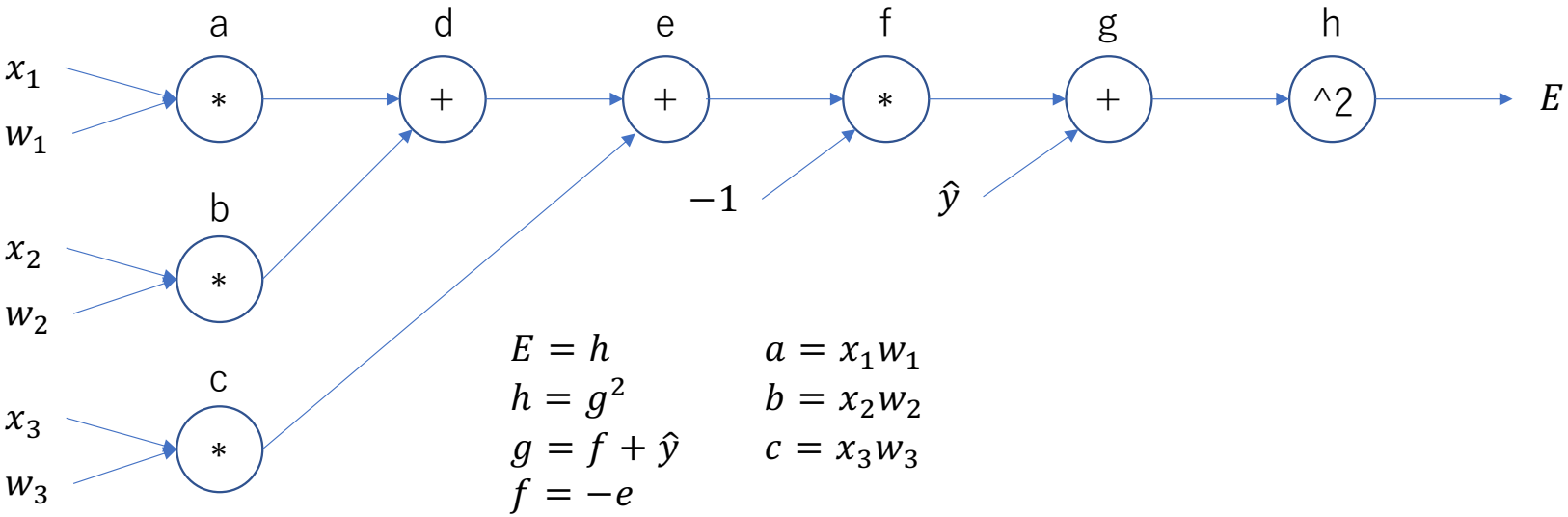
$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial t}\frac{\partial t}{\partial x} = (2t)a = 2a(ax + b)$$

# Back Propagation (1/2)



## Computational Graph

$$E = (\hat{y} - y)^2 = (\hat{y} - x_1 w_1 - x_2 w_2 - x_3 w_3)^2$$



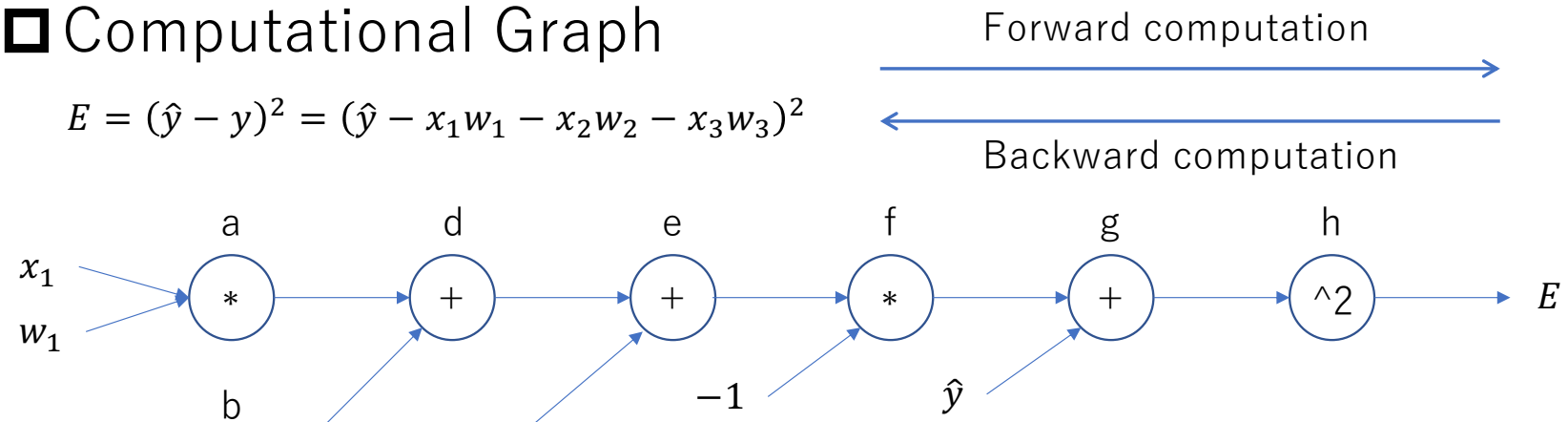

$$E = h \qquad a = x_1 w_1$$
$$h = g^2 \qquad b = x_2 w_2$$
$$g = f + \hat{y} \qquad c = x_3 w_3$$
$$f = -e$$
$$e = d + c$$
$$d = a + b$$

# Back Propagation (2/2)



## Computational Graph

$$E = (\hat{y} - y)^2 = (\hat{y} - x_1 w_1 - x_2 w_2 - x_3 w_3)^2$$




$$E = h$$
$$h = g^2$$
$$g = f + \hat{y}$$
$$f = -e$$
$$e = d + c$$
$$d = a + b$$

$$a = x_1 w_1$$
$$b = x_2 w_2$$
$$c = x_3 w_3$$

$$\frac{\partial E}{\partial h} = 1, \frac{\partial h}{\partial g} = 2g, \frac{\partial g}{\partial f} = 1, \frac{\partial f}{\partial e} = -1, \frac{\partial e}{\partial d} = 1, \frac{\partial d}{\partial a} = 1, \frac{\partial a}{\partial w_1} = x_1$$

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial h}\frac{\partial h}{\partial g}\frac{\partial g}{\partial f}\frac{\partial f}{\partial e}\frac{\partial e}{\partial d}\frac{\partial d}{\partial a}\frac{\partial a}{\partial w_1} = 1 * 2g * 1 * (-1) * 1 * 1 * x_1$$

$$= -2gx_1 = -2(f + \hat{y})x_1 = -2x_1(\hat{y} - e)$$
$$= -2x_1\big(\hat{y} - (d + c)\big) = -2x_1\big(\hat{y} - (a + b + c)\big)$$
$$= -2x_1\big(\hat{y} - (x_1 w_1 + x_2 w_2 + x_3 w_3)\big)$$
$$= -2x_1(\hat{y} - x_1 w_1 - x_2 w_2 - x_3 w_3)$$